

Zusatzaufgabe: Terraform + GitHub Actions nachbauen und erweitern

Baut den heutigen Ablauf in einem eigenen Repository oder in einem Test-Branch nach. Ziel ist, dass ihr versteht, wie Terraform nicht nur lokal, sondern auch über GitHub Actions ausgeführt werden kann.

Aufgabe:

- Erstellt oder nutzt ein Terraform-Projekt mit `main.tf`, `variables.tf` und `outputs.tf`.
- Erstellt mit Terraform mindestens eine EC2-Instanz.
- Definiert einen Output für die Public IP-Adresse.
- Erstellt in GitHub Secrets eure AWS-Zugangsdaten.
- Erstellt einen GitHub Actions Workflow.
- Der Workflow soll beim Push starten.
- Der Workflow soll den Code auschecken.
- Der Workflow soll Terraform installieren.
- Der Workflow soll AWS Credentials konfigurieren.
- Der Workflow soll `terraform init` ausführen.
- Der Workflow soll `terraform apply -auto-approve` ausführen.
- Lest danach mit `terraform output` die Public IP aus.
- Baut optional einen SSH-Step ein, der sich mit der EC2-Instanz verbindet.
- Erstellt zusätzlich einen manuellen Destroy-Workflow oder einen Destroy-Job mit `workflow_dispatch`.

Erweiterungsideen:

- Trennt Apply und Destroy in zwei verschiedene Workflow-Dateien.
- Baut eine Slack-Benachrichtigung ein, wenn Apply erfolgreich war.
- Baut eine Slack-Benachrichtigung ein, wenn Destroy ausgeführt wurde.
- Ersetzt `sleep 10` durch eine kleine Warteschleife, die prüft, ob SSH erreichbar ist.
- Erstellt eine Security Group mit Terraform und öffnet gezielt Port 22 oder Port 80.
- Deployt eine einfache `index.html` per SSH auf die EC2-Instanz.
- Erstellt einen Schedule, der die Infrastruktur morgens erstellt und abends löscht.
- Lagert mehr Werte in Variablen aus, zum Beispiel Instanztyp, Region oder Anzahl der Maschinen.
- Gebt mehrere Outputs aus, zum Beispiel Public IP, Instance ID und Public DNS.

Ganz wichtig: Wenn ihr fertig seid, führt unbedingt euren Destroy-Workflow oder `terraform destroy` aus, damit eure AWS-Ressourcen gelöscht werden und keine unnötigen Kosten entstehen.